KARMA Supplementary Appendix

1 KARMA Operator Descriptions

Table 1: KARMA operator descriptions

Operator	Output	Description
EntityLink $(q, \mathcal{G})$	E_q=$\{e_1, \ldots, e_k\}$	DBpedia Spotlight (confidence threshold 0.5) with recovery linking returns DBpedia resource IRIs; these entities seed schema projection, SPARQL generation, and entity-centered text retrieval.
SchemaProject (E_q, q)	$\mathcal{S}_q$=$\langle P_q, A_q \rangle$	Builds the projected schema for q: P_q contains instantiated (e, p) pairs used as coverage evidence, while A_q stores annotation metadata such as class, range, type, or datatype fields. Bridge candidates may be shown in traces, but coverage still depends on instantiated KG evidence, not on annotations alone.
Covers $(\mathcal{S}_q, q)$	**true**/**false**; covered, uncovered; next branch	Checks whether each relation in Rel(q) has a coverage-compatible predicate in P_q on the required linked entity. The reported covered and uncovered relations determine whether the algorithm continues with SPARQL, decomposition, or text retrieval.
Text2SPARQL $(q, \mathcal{S}_q, \mathcal{G})$	SPARQL query, bindings R, answer a or $\emptyset$	A schema-grounded prompt asks the LLM to generate a SPARQL query from q and $\mathcal{S}_q$; the KG returns bindings and any answer. Non-empty answers return or enter M; empty results leave the step unresolved.
Decompose (Q)	D=$(\{q_i\}, \prec)$	A decomposition prompt asks the LLM to return a dependency-ordered sub-question DAG. The order tells the system which sub-answers must be produced before later questions can be grounded.
Ground (q_i, M)	$\hat{q}_i$	Deterministic substitution produces a concrete sub-question by replacing dependencies with answers already stored in M.
TextRAG $(q, E_q, \mathcal{T})$	relevant passages, answer a or a_i	Retrieval supplies entity-seeded passages when available; an answer prompt asks the LLM to answer q using those passages after failed coverage, empty SPARQL results, or empty synthesis.
M and Synthesize (Q, M)	$M[q_i] \leftarrow a_i$; final answer a	M records sub-question answers. A synthesis prompt asks the LLM to combine M into the final answer; if it returns $\emptyset$, the algorithm calls final TextRAG.

As in Table 2, SchemaProject rows display $\mathcal{S}_q = \langle P_q, A_q \rangle$: P_q contains coverage-bearing instantiated predicates, while A_q contains annotations such as class metadata. Candidate bridge paths are shown only when retained for routing. Covers rows show the Boolean result, covered and uncovered relation checks, and the resulting branch.

Table 2: End-to-end trace of KARMA on Q="What did the spouse of Andrea Dotti win a Tony for?". SCHEMAPROJECT reports annotation metadata, such as class, and instantiated predicates; only instantiated predicates provide coverage evidence. The table reports the Boolean COVERS result and the branch subsequently taken by the KARMA algorithm. $\hat{q}$ denotes the instantiated form of a sub-question.

Step	Operator call	Output / state
Top level — COVERS fails, fall through to decomposition		
1	ENTITYLINK(Q)	E={`dbr:Andrea_Dotti_(psychiatrist)`}
2	SCHEMAPROJECT(E, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Person`; P_Q: {`dbo:spouse`, `dbo:birthDate`, ...} **false**; covered={*spouse* -> `dbo:spouse`}
3	COVERS($\mathcal{S}, Q$)	uncovered={*win a Tony for* -> no coverage-compatible instantiated predicate} next branch=**DECOMPOSE**
4	DECOMPOSE(Q)	D={q_1, q_2}, $q_1 \rightarrow q_2$; $M \leftarrow \emptyset$ q_1="Who is the spouse of Andrea Dotti?" q_2="What did [q_1] win a Tony for?"
*Iterate q_1 (M=$\emptyset$) ⇒ **KG-only***		
5	GROUND(q_1, M)	$\hat{q}_1$="Who is the spouse of Andrea Dotti?"
6	ENTITYLINK($\hat{q}_1$)	E_{q_1}={`dbr:Andrea_Dotti_(psychiatrist)`}
7	SCHEMAPROJECT($E_{q_1}, \hat{q}_1$)	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Person`; P_{q_1}: {`dbo:spouse`, `dbo:birthDate`, ...} **true**; covered={*spouse* -> `dbo:spouse`}
8	COVERS($\mathcal{S}_{q_1}, \hat{q}_1$)	uncovered=$\emptyset$; next branch=**SPARQL**
9	TEXT2SPARQL	`SELECT ?s WHERE {dbr:Andrea_Dotti_(psychiatrist) dbo:spouse ?s}` $\rightarrow a_{q_1}$ = *Audrey Hepburn*; $M[q_1] \leftarrow$ *Audrey Hepburn*
Iterate q_2 ($M[q_1]$= Audrey Hepburn*) ⇒ **text** (coverage fails)*		
10	GROUND(q_2, M)	$\hat{q}_2$="What did Audrey Hepburn win a Tony for?"
11	ENTITYLINK($\hat{q}_2$)	E_{q_2}={`dbr:Audrey_Hepburn`} (`dbo:Person`)
12	SCHEMAPROJECT($E_{q_2}, \hat{q}_2$)	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2}: {`dbo:spouse`, `dbo:occupation`, ...} **false**; covered=$\emptyset$
13	COVERS($\mathcal{S}_{q_2}, \hat{q}_2$)	uncovered={*win a Tony for* -> no coverage-compatible instantiated predicate} next branch=**TEXT**
14	TEXTRAG($\hat{q}_2, E_{q_2}, \mathcal{T}$)	retrieved text: "...recipient of the Cecil B. DeMille Award in 1990. She also won the 1954 Tony Award for ***Best Performance by a Leading Actress in a Play in Ondine***, and received a Special Tony Award in 1968. ..." $\rightarrow a_{q_2}$ = *Best Performance by a Leading Actress in a Play in Ondine*
15	SYNTHESIZE(Q, M)	*for Best Performance by a Leading Actress in a Play in Ondine*

2 End-to-end execution trace

Table 3: Trace-style examples of KARMA execution regimes selected from judged evaluation traces. Each block follows the same presentation pattern as Table 2: schema projection rows separate coverage-bearing predicates from annotations, and coverage rows report covered relations, uncovered relations, and the next branch. Text2SPARQL rows omit shared prefixes and abbreviate DBpedia IRIs with `dbr:`.

Step	Operator call	Output / state
Route: *Direct SPARQL* **Question:** "Where was the director of film *Thomas Jefferson (Film)* born?"		
1	EntityLink(Q)	E_Q={e_0=`dbr:Thomas_Jefferson_(film)`}.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Film`; P_Q: {`dbo:director`, ...}; bridge path: `dbo:director`→`dbo:birthPlace`.
3	Covers ($\mathcal{S}_Q, Q$)	**true**; covered={*director* -> `dbo:director`, *born* -> `dbo:director` → `dbo:birthPlace`}; uncovered=∅; next branch=**SPARQL**.

4	Text2SPARQL ($Q, \mathcal{S}_Q, \mathcal{G}$)	`SELECT ?birthPlace`
		`WHERE {`
		`dbr:Thomas_Jefferson_(film) dbo:director ?person .`
		`?person dbo:birthPlace ?birthPlace .`
		`}`
		`LIMIT 100`
		R={`dbr:Brooklyn`}.
		a=*Brooklyn*.

Route: *Direct SPARQL* **Question:** "Which film has the director born later, *House Of Blackmail* or *Taxi Zum Klo*?"		
1	EntityLink(Q)	E_Q={e_1, e_2}; e_1=`dbr:House_of_Blackmail`; e_2=`dbr:Taxi_zum_Klo`.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; for each $e \in E_Q$, A_Q: class=`dbo:Film`; P_Q: {`dbo:director`, ...}; bridge path: `dbo:director`→`dbo:birthDate`.
3	Covers ($\mathcal{S}_Q, Q$)	**true**; covered={*director* -> `dbo:director`, *born later* -> `dbo:director` → `dbo:birthDate`}; uncovered=∅; next branch=**SPARQL**.

4	Text2SPARQL ($Q, \mathcal{S}_Q, \mathcal{G}$)	`SELECT ?film`
		`WHERE {`
		`VALUES ?film { dbr:House_of_Blackmail dbr:Taxi_zum_Klo }`
		`?film dbo:director ?director .`
		`?director dbo:birthDate ?birthDate .`
		`}`
		`ORDER BY DESC(?birthDate)`
		`LIMIT 1`
		a=*Taxi Zum Klo*.

Route: *Decomp: all SPARQL* **Question:** "Who was born first out of Marco De Luigi and Fabio Maistro?"		
1	EntityLink(Q)	E_Q={e_1, e_2}; e_1=`dbr:Marco_de_Luigi`; e_2=`dbr:Fabio_Maistro`.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; for each $e \in E_Q$, A_Q: class=`dbo:Person`; P_Q: {`dbo:birthDate`, ...}.
3	Covers ($\mathcal{S}_Q, Q$)	covered={*born* -> `dbo:birthDate` for both entities}; uncovered={*born first* -> no accepted whole-question comparison route}; next branch=**DECOMPOSE**.
4	Decompose(Q)	LLM decomposition; D=($\{q_1, q_2\}, \emptyset$), ordered as q_1, q_2; q_1="When was Marco De Luigi born?"; q_2="When was Fabio Maistro born?"; M←∅.
5	Ground(q_1, M)	$\hat{q}_1$="When was Marco De Luigi born?".
6	EntityLink($\hat{q}_1$)	E_{q_1}={e_1=`dbr:Marco_de_Luigi`}.
7	SchemaProject ($E_{q_1}, \hat{q}_1$)	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Person`; P_{q_1}: {`dbo:birthDate`, ...}.
8	Covers ($\mathcal{S}_{q_1}, \hat{q}_1$)	**true**; covered={*born* -> `dbo:birthDate`}; uncovered=∅; next branch=**SPARQL**.

9	Text2SPARQL ($\hat{q}_1, \mathcal{S}_{q_1}, \mathcal{G}$)	`SELECT ?birthDate`
		`WHERE {`
		`dbr:Marco_de_Luigi dbo:birthDate ?birthDate .`
		`}`
		`LIMIT 100`
		R={1978-03-21}; $M[q_1]$←*March 21, 1978*.

10	Ground(q_2, M)	$\hat{q}_2$="When was Fabio Maistro born?".
11	EntityLink($\hat{q}_2$)	E_{q_2}={e_2=`dbr:Fabio_Maistro`}.

continued on next page

Table 3: Trace-style examples of KARMA execution regimes (continued).

Step	Operator call	Output / state
12	SchemaProject $(E_{q_2}, \hat{q}_2)$	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2}: {`dbo:birthDate`, ...}.
13	Covers $(\mathcal{S}_{q_2}, \hat{q}_2)$	**true**; covered={*born* -> `dbo:birthDate`}; uncovered=$\emptyset$; next branch=**SPARQL**.

14	Text2SPARQL $(\hat{q}_2, \mathcal{S}_{q_2}, \mathcal{G})$	`SELECT ?birthDate`
		`WHERE {`
		`dbr:Fabio_Maistro dbo:birthDate ?birthDate .`
		`}`
		`LIMIT 100`
		R={1998-04-05}; $M[q_2] \leftarrow$ *April 5, 1998*.

15	Synthesize(Q, M)	LLM synthesis compares $M[q_1]$ and $M[q_2]$; a=*Marco De Luigi*.
Route: *Decomp: SPARQL+text* **Question:** "Where was the director of film *Age-Old Friends* born?"		
1	EntityLink(Q)	E_Q={e_0=`dbr:Age-Old_Friends`}.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Film`; P_Q: {`dbo:director`, ...}; candidate bridge path: `dbo:director`→`dbo:birthPlace`.
3	Covers $(\mathcal{S}_Q, Q)$	**false**; covered={*director* -> `dbo:director`}; uncovered={*born* -> unavailable bridge `dbo:director` → `dbo:birthPlace`}; next branch=**DECOMPOSE**.
4	Decompose(Q)	LLM decomposition; D=($\{q_1, q_2\}, q_1 \rightarrow q_2$); q_1="Who directed Age-Old Friends?"; q_2="Where was [q_1] born?"; $M \leftarrow \emptyset$.
5	Ground(q_1, M)	$\hat{q}_1$="Who directed Age-Old Friends?".
6	EntityLink$(\hat{q}_1)$	E_{q_1}={e_0}.
7	SchemaProject $(E_{q_1}, \hat{q}_1)$	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Film`; P_{q_1}: {`dbo:director`, ...}.
8	Covers $(\mathcal{S}_{q_1}, \hat{q}_1)$	**true**; covered={*directed* -> `dbo:director`}; uncovered=$\emptyset$; next branch=**SPARQL**.

9	Text2SPARQL $(\hat{q}_1, \mathcal{S}_{q_1}, \mathcal{G})$	`SELECT ?director`
		`WHERE {`
		`dbr:Age-Old_Friends dbo:director ?director .`
		`}`
		`LIMIT 100`
		R={`dbr:Allan_Kroeker`}; $M[q_1] \leftarrow$ *Allan Kroeker*.

10	Ground(q_2, M)	$\hat{q}_2$="Where was Allan Kroeker born?".
11	EntityLink$(\hat{q}_2)$	E_{q_2}={e_1=`dbr:Allan_Kroeker`}.
12	SchemaProject $(E_{q_2}, \hat{q}_2)$	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2} does not contain `dbo:birthPlace`.
13	Covers $(\mathcal{S}_{q_2}, \hat{q}_2)$	**false**; covered=$\emptyset$; uncovered={*born* -> no coverage-compatible `dbo:birthPlace`}; next branch=**TEXT**.
14	TextRAG $(\hat{q}_2, E_{q_2}, \mathcal{T})$	retrieved text: "Allan Kroeker (born February 29, 1960 in ***Winnipeg, Manitoba***) is a Canadian film and television director ..." → a_{q_2}=*Winnipeg, Manitoba*; $M[q_2] \leftarrow$ *Winnipeg, Manitoba*.
15	Synthesize(Q, M)	LLM synthesis; a=*Winnipeg, Manitoba*.

continued on next page

Table 3: Trace-style examples of KARMA execution regimes (continued).

Step	Operator call	Output / state
Route: *Text only* **Question:** "Why did John Middleton Murry's wife die?"		
1	EntityLink(Q)	E_Q={e_0=`dbr:John_Middleton_Murry`}.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Person`; P_Q contains neither `dbo:spouse` nor `dbo:deathCause`.
3	Covers ($\mathcal{S}_Q, Q$)	covered=$\emptyset$; uncovered={*wife* -> no observed `dbo:spouse`, *die* -> no observed `dbo:deathCause`}; next branch=**DECOMPOSE**.
4	Decompose(Q)	LLM decomposition; D=($\{q_1, q_2\}, q_1 \rightarrow q_2$); q_1="Who was John Middleton Murry's wife?"; q_2="Why did [q_1] die?"; $M \leftarrow \emptyset$.
5	Ground(q_1, M)	$\hat{q}_1$="Who was John Middleton Murry's wife?".
6	EntityLink($\hat{q}_1$)	E_{q_1}={e_0}.
7	SchemaProject ($E_{q_1}, \hat{q}_1$)	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Person`; P_{q_1} does not contain `dbo:spouse`.
8	Covers ($\mathcal{S}_{q_1}, \hat{q}_1$)	**false**; covered=$\emptyset$; uncovered={*wife* -> no coverage-compatible `dbo:spouse`}; next branch=**TEXT**.
9	TextRAG ($\hat{q}_1, E_{q_1}, \mathcal{T}$)	retrieved text: "...Murry is best remembered for his association with ***Katherine Mansfield***, whom he married in 1918 as her second husband ..." $\rightarrow a_{q_1}$=*Katherine Mansfield*; $M[q_1] \leftarrow$*Katherine Mansfield*.
10	Ground(q_2, M)	$\hat{q}_2$="Why did Katherine Mansfield die?".
11	EntityLink($\hat{q}_2$)	E_{q_2}={e_1=`dbr:Katherine_Mansfield`}.
12	SchemaProject ($E_{q_2}, \hat{q}_2$)	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2} does not contain `dbo:deathCause`.
13	Covers ($\mathcal{S}_{q_2}, \hat{q}_2$)	**false**; covered=$\emptyset$; uncovered={*die* -> no coverage-compatible `dbo:deathCause`}; next branch=**TEXT**.
14	TextRAG ($\hat{q}_2, E_{q_2}, \mathcal{T}$)	retrieved text: "Mansfield was diagnosed with extrapulmonary ***tuberculosis*** in 1917; the disease claimed her life at the age of 34." $\rightarrow a_{q_2}$=*tuberculosis*; $M[q_2] \leftarrow$*tuberculosis*.
15	Synthesize(Q, M)	LLM synthesis; a=*Tuberculosis*.

3 Prompt Catalogue

For each prompt, we report its identifier, execution stage, and condition of use. The boxes below render runtime templates with placeholders; dynamic values such as candidate lists and retrieved passages are inserted at execution time.

Table 4: Prompt catalogue summary.

ID	Stage	Prompt	Condition of use
P1	Entity linking	Entity candidate disambiguation	Only when fallback entity search returns multiple DBpedia candidates for an unlinked mention.
P2	Routing	Schema-constrained property extraction	Only when deterministic keyword/schema matching is inconclusive; candidate properties are passed back to deterministic routing and ASK checks.
P3	Decomp.	Question decomposition	Used after the direct symbolic shortcut fails and decomposition is needed.
P4	Decomp.	Early stopping / accumulated facts	Used during decomposition to decide whether accumulated facts already answer the original question.
P5	SPARQL	SPARQL generation	Used for direct SPARQL and routed sub-question SPARQL.
P6	SPARQL	SPARQL repair	Used once when a generated SPARQL query errors or returns empty results.
P7	SPARQL	SPARQL result verbalization	Used to convert SPARQL result rows into a concise final answer.
P8	Text fallback	Whole-question answer prompt	Used when a whole question is routed to text or symbolic execution cannot answer.
P9	Text fallback	Sub-question answer prompt	Used when an individual decomposed sub-question is routed to text.
P10	Synthesis	Final answer synthesis	Used to synthesize the final answer from resolved sub-question facts.

continued on next page

ID	Stage	Prompt	Condition of use
J1	Judge	LLM-as-judge system and user prompts	Used only for evaluation records not handled by deterministic empty-answer, exact-alias, or containment pre-checks.

3.1 Runtime KARMA prompts

The prompts below are the active LLM templates used by the reported KARMA runtime. Prompt P2 is a fallback property-enrichment prompt: it proposes candidate DBpedia properties, but the route is still determined by schema matching and ASK verification.

P1. Entity candidate disambiguation

```
Given this question: "{question}"
Which entity best matches the mention "{mention}"?

{candidate_list}

Reply with ONLY the number (1-{num_candidates}) of the best match, or "none" if no candidate fits.
```

P2. Schema-constrained property extraction

```
Given this question and DBpedia schema, which properties are needed to answer it?

{schema}

QUESTION: {question}

Output a comma-separated list of property names (no dbo:/dbp: prefix).
Only output properties listed above. For 2-hop paths (e.g., "director's birthdate"),
list both: director, birthDate
If no properties can help, output: NONE

Properties:
```

P3. Decomposition prompt

```
Decompose this multi-hop question into independent sub-questions.

QUESTION: {question}
ENTITIES: {entity_names}
SCHEMA SUMMARY: {schema_summary}
{type_hint}
Rules:
- Each sub-question should ask for ONE fact
- If a sub-question needs the answer from another, mark the dependency
- For comparison questions: extract the same property for each entity (parallel, no dependencies)
- For bridge questions: first find the intermediate entity, then ask about its property
- If the question is simple (single fact lookup), return exactly 1 sub-question with no dependencies

Return JSON:
{
  "sub_questions": ["sub-Q1 text", "sub-Q2 text", ...],
  "dependencies": [[dependent_idx, dependency_idx], ...]
}

Example - Bridge: "When was the director of Inception born?"
{
  "sub_questions": ["Who directed Inception?", "When was [Q0] born?"],
  "dependencies": [[1, 0]]
}

Example - Comparison: "Who was born first, the director of X or Y?"
{
  "sub_questions": ["Who directed X?", "Who directed Y?",
    "When was [Q0] born?", "When was [Q1] born?"],
  "dependencies": [[2, 0], [3, 1]]
}

Example - Simple: "Who directed Inception?"
{
  "sub_questions": ["Who directed Inception?"],
  "dependencies": []
}

Example - Nested relative clause (3-hop): "When was the region north of where X is located created?"
{
```

```
  "sub_questions": ["Where is X located?", "What region is immediately north of [Q0]?", "When was [Q1] created?"],
  "dependencies": [[1, 0], [2, 1]]
}

Example - Passive voice: "Who is played by the director of Film X in Film Y?"
{
  "sub_questions": ["Who directed Film X?", "What role did [Q0] play in Film Y?"],
  "dependencies": [[1, 0]]
}

Example - Possessive chain: "What is the capital of the country where X's creator was born?"
{
  "sub_questions": ["Who created X?", "In which country was [Q0] born?", "What is the capital of [Q1]?"],
  "dependencies": [[1, 0], [2, 1]]
}

Additional rules for complex questions:
- Preserve active/passive voice direction: "played BY X" -> ask "What did [Q0] play?" NOT "Who played [Q0]?"
- For possessive chains (X's Y's Z), decompose from innermost entity outward
- Each sub-question MUST retain qualifying context (where/who/that clauses) from the original

JSON only, no explanation:
```

P4. Early stopping prompt

```
We pre-parsed the question into a list of dependencies, and the dependencies are sorted in a topological order, below
     is the question, the information summary, and the decomposed dependencies:

Question: {question}

Available Information:
{info_summary}

Decomposed dependencies:
{dependencies}

Current dependency to be answered:
{current_dependency}

Please analyze the question and the information summary, and the decomposed dependencies, and answer the following
     questions:
Please analyze:
1. Can the question be answered completely with this information? (Yes/No)
2. Summarize our current understanding based on available information.

Please format your response as a JSON object with these keys:
- "can_answer": boolean
- "current_understanding": string
```

P5. SPARQL generation prompt

```
You are a SPARQL expert for DBpedia. Generate a query to answer the question.

{schema}
{known_facts_block}
QUESTION: {question}

RULES:
1. Use the EXACT entity URIs provided in the schema
2. For multi-hop queries, chain triple patterns using dbo: properties (they return URIs, not strings).
   Use the "2-Hop chains" section in the schema to identify valid chains.
   Example: if schema shows dbo:director -> Person: birthDate, write:
   ?film dbo:director ?person . ?person dbo:birthDate ?date .
3. Prefer dbo: properties for multi-hop joins; only use dbp: for single-hop leaf values
4. Always add LIMIT 100
5. For comparison questions (who/which was born first, etc.), return data for BOTH entities so results can be compared
6. If an entity URI contains special characters (apostrophes, commas, parentheses), use full IRI syntax <http://
     dbpedia.org/resource/...> instead of dbr: prefix
7. If known facts provide entity URIs, use those URIs directly in your query
8. Generate ONLY the SPARQL query, no explanation
```

P6. SPARQL repair prompt

```
Your previous SPARQL query returned {problem}.

Previous query:
{sparql}

Fix the query using the schema below.

{schema}
```

```
QUESTION: {question}

Generate ONLY the corrected SPARQL query:
```

P7. SPARQL result verbalization prompt

```
Given the question and SPARQL query results from DBpedia, provide the answer.
Base your answer ONLY on the SPARQL results shown below, not on your own knowledge.
For comparison questions: if data is missing for one of the entities being compared, say "could not determine" instead
      of guessing.

Question: {question}
SPARQL Results:
{results}

End your response with FINAL ANSWER: <your answer>
```

P8. Whole-question text fallback prompt

```
Answer this question using the sources below.

Question: {question}

{kg_facts}Wikipedia text:
{context}

Instructions:
- Knowledge graph facts (if present) are verified structured data -- trust them
- Use Wikipedia text to fill gaps not covered by KG facts
- Show brief step-by-step reasoning
- End with exactly: FINAL ANSWER: <answer>
```

P9. Sub-question text fallback prompt

```
Answer this sub-question using the sources below.

Sub-question: {question}

{kg_facts}Wikipedia text:
{context}

Instructions:
- Knowledge graph facts (if present) are verified structured data -- trust them
- Use Wikipedia text to fill gaps not covered by KG facts
- Give a brief, factual answer
- End with exactly: FINAL ANSWER: <answer>
```

P10. Final answer synthesis prompt

```
You must give ONLY the direct answer in the most concise way possible.
DO NOT explain or provide any additional context.
If the answer is a simple yes/no, just say "Yes." or "No."
If the answer is a name, just give the name.
If the answer is a date, just give the date.
If the answer is a number, just give the number.
If the answer requires a brief phrase, make it as concise as possible.

IMPORTANT -- the sub-questions below form a reasoning chain:
- Earlier sub-questions find intermediate entities; later ones use those entities.
- The LAST sub-question's answer is usually what the original question asks for.
- Do NOT return an intermediate entity when a later sub-question has resolved the final answer.
- If a sub-question returns a list, pick the item that satisfies the condition in the original question.
- For yes/no questions with multiple sub-questions, combine ALL sub-answers (both must be true for "yes").
- If any sub-question is marked [UNKNOWN], the chain is incomplete. Do NOT fabricate
  a final answer by skipping the gap -- respond with: FINAL ANSWER: unknown

[Inserted only when is_comparison=True]
When comparing countries or nationalities:
- If answers contain city or state names, identify which country they belong to
- Compare at the country level, not city/state level
- Historical states (East Germany, Ottoman Empire) map to their modern successor

Question: {question}

Information Summary:
{info_summary}

End your response with exactly: FINAL ANSWER: <your answer>
```

3.2 Evaluation judge prompt

Prompt J1 is not a KARMA runtime prompt. It is used only by the evaluation script after deterministic checks have failed to decide the record: empty predictions are false, exact gold-or-alias matches are true, and a normalized gold-in-prediction containment pre-check can return true before the LLM call. The LLM judge applies the benchmark-equivalence rules shown below, including alias, containment, and selected location-normalization cases; it is stricter than free-form semantic similarity, but it is not an exact-match metric.

J1. LLM-as-judge prompt

```
System:
You are a strict evaluation judge for a QA benchmark. Determine if two answers refer to the exact same entity or fact.
      Precision matters more than recall -- when in doubt, answer no.

User:
Determine if the predicted answer is semantically equivalent to the gold answer.

Question: {question}
Gold answer: {gold}
Predicted answer: {pred}

Matching rules (answer YES):
- Same entity/fact with different surface forms (e.g. "July 16, 2013" vs "2013-07-16")
- Gold contained in a more specific predicted answer (e.g. "Munich" vs "Nymphenburg Palace, Munich")
- Added title/honorific/disambiguator (e.g. "Henry III" vs "Henry III, Holy Roman Emperor")
- Full formal name of same person (e.g. "Giovanni Mario" vs "Giovanni Mario de Candia")
- Administrative region containing the exact gold location (e.g. gold "Theni" vs pred "Theni district")
- Unambiguous well-known city with or without country (e.g. gold "Florence, Italy" vs pred "Florence")
- Nationality adjective matching country name (e.g. gold "Italy" vs pred "Italian")

Rejection rules (answer NO):
- Wrong, empty, or completely different entity
- Predicted is broader/vaguer than gold (e.g. gold "Taipei Medical University Hospital" vs pred "Taipei")
- Year only when gold has full date (e.g. gold "December 25, 1757" vs pred "1757")
- Related but distinct entities (e.g. gold "Hollywood" vs pred "Los Angeles")
- Same first name but different person (e.g. gold "Paris" vs pred "Paris Hilton")
- Predicted lists multiple answers and gold is NOT clearly among them
- Completely different location even if in same region (e.g. gold "Theni" vs pred "Mallingapuram")

First, explain in ONE sentence why the answers match or don't.
Then write your verdict on a new line: VERDICT: yes or VERDICT: no
```

4 Cross-benchmark routing breakdown

The main paper plots the Q-level and Sub-Q-level routing distributions. Table 5 gives the underlying routing counts.

Table 5: Per-benchmark routing statistics over 1,000 questions each. Direct SPARQL: whole question answered by Text2SPARQL without decomposition. Sub-Qs: total sub-questions produced when decomposition fires. Sub-SPARQL%: percentage of generated sub-questions resolved symbolically. Any-SPARQL%: fraction of original questions in which at least one sub-question hit the SPARQL path. Routing distributions on 2WikiMHQA and MuSiQue are stable across runs (any-SPARQL drift $\leq 2.6\%$); HotpotQA is more variable (15.4%), reflecting LLM stochasticity in the coverage check when KG support is sparse.

Metric	2WikiMHQA	HotpotQA	MuSiQue
Direct SPARQL (%)	29.5	6.1	2.4
Decomposition fired (%)	70.1	88.7	96.5
Sub-questions generated	1544	1807	2508
Sub-SPARQL (%)	40.5	7.6	18.0
Any-SPARQL touch (%)	**70.0**	**17.4**	**42.6**

Table 6: Failure clusters across 589 judged-incorrect questions; counts per benchmark and share of total.

#	Cluster	2Wiki	HPQA	MuSiQue	Total	%
I	KG forces text, text fails	61	74	102	237	40.2
II	Sub-question returns wrong fact	2	6	82	90	15.3
III	Routing bypass on multi-hop	22	30	13	65	11.0
IV	Under-decomposition	3	0	85	88	14.9
V	Over-decomposition	—	—	38	38	6.5
VI	Comparison-logic error	18	16	—	34	5.8
VII	Entity-link / null sub-Q	0	8	—	8	1.4
VIII	Hedging / refusal	0	0	14	14	2.4
IX	DBpedia–Wikipedia format gap	0	5	—	5	0.8
X	Chain-text-miss intermediate	0	0	10	10	1.7
	Total	106	139	344	**589**	**100**

5 Error Analysis

To understand *why* KARMA fails, we classify the 589 judged-incorrect questions across the three benchmarks (106/1000 on 2Wiki, 139/1000 on HotpotQA, 344/1000 on MuSiQue; aggregate fail rate 19.6%) by inspecting their full pipeline traces (routing decisions, sub-question logs, retrieved chunks). All counts below are verified against the raw judged JSONL (judge: GEMMA3:12B). MuSiQue's failure rate (34.4%) is 3.2× the 2Wiki rate, consistent with its higher hop count and lower KG connectivity (Table 7); we do not attempt to fully decouple hop depth from other inter-benchmark construction differences.

Failure taxonomy. Table 6 groups the 589 failures into ten clusters. Six dominate ($\geq 5\%$ of failures); we describe each below.

Cluster I: KG forces text, text fails (40.2%, the dominant mode). The KG signals absence of a 2-hop path; the question falls back to text retrieval, which then fails for benchmark-specific reasons. The trigger is shared, but the downstream failure differs: *(i)* on 2Wiki (61 cases), the system retrieves the right Wikipedia page but the predicted string refers to the same concept under a different label (e.g. "Ottoman Empire" vs. "Turkey"), so EM/F1 string-match fails; *(ii)* on HotpotQA (74 cases), the BM25 + reranker pipeline promotes a topically similar but wrong Wikipedia page (e.g. the director's biography instead of the film page), all 74 exhibiting topically-related but incorrect retrieval; *(iii)* on MuSiQue (102 cases), DBpedia connectivity collapses beyond two hops (Table 7), so text must traverse the entire chain unaided, which Wikipedia pages do not support.

Cluster IV: under-decomposition (14.9%, almost entirely MuSiQue). The decomposition LLM produces fewer sub-questions than the hop count: 46 cases of 4-hop $\rightarrow$ 3 sub-Qs, 30 of 3-hop $\rightarrow$ 2, 7 of 4-hop $\rightarrow$ 2, 2 of 2-hop $\rightarrow$ 1. About 80% of these 88 cases co-occur with all-text retrieval: when the KG cannot help, decomposition errors compound. We leave prompt revisions targeting this pattern to future work.

Cluster II: sub-question returns wrong fact (15.3%, dominant on MuSiQue). A sub-question executes but retrieves the wrong answer. We focus on MuSiQue's 82 cases, where the pattern is sharpest; the remaining 8 (2 on 2Wiki, 6 on HotpotQA) were not separately re-classified. On the 82 MuSiQue cases, 91.5% (75/82) have the gold entity *absent from the entire retrieved chain*—only 7/82 are genuine final-synthesis errors, suggesting the bottleneck is retrieval, not synthesis: improvements to the synthesis prompt will not help if the gold fact was never retrieved.

Table 7: MuSiQue failure rate vs. DBpedia chain connectivity by hop type. Connectivity = fraction of questions whose entity chain is fully traversable in DBpedia.

Hop type	Fail %	DBpedia connectivity
2hop	28.6	32.8
3hop1	24.7	14.4
3hop2	45.2	1.4
4hop1	55.6	2.8
4hop2	**92.6**	0.0
4hop3	58.1	0.0

Cluster III: routing bypass (11.0%, universal across benchmarks). Multi-hop questions are answered by SPARQL alone because routing matches a surface predicate (e.g. "director" → `dbo:director`) and returns early, skipping decomposition (22/30/13 on the three benchmarks). A single guard—force decomposition consideration when the question is multi-hop—would, in principle, unblock decomposition for all 65 cases; the recoverable subset is bounded by sub-question success rates and is not directly measured here.

Cluster V: over-decomposition (6.5%, MuSiQue only). The opposite failure of Cluster IV: 14/38 cases follow a 2-hop → 3+ sub-question pattern in which a phantom sub-question introduces retrieval noise that propagates into the final answer; a further 5–7 cases are 4-hop3 questions split into 5–6 sub-questions, and the remaining cases are over-decomposed by other patterns we do not separately enumerate. The fix points to bidirectional decomposition guidance (produce roughly N sub-questions for an N-hop question), not just a stricter floor.

Cluster VI: comparison logic (5.8%). Apparent reasoning errors that on inspection are date/string-format or value-multiplicity mismatches—for example, a film with multiple listed release dates (1948, 1968, 1991) compared against a single-date counterpart (1988) causes the comparison prompt to flip the answer; nationality-equivalence cases (e.g. "Greek and Italian" vs. "Greek") exhibit a similar pattern. A canonical date and nationality mapper applied prior to comparison addresses these cases.

Hop-depth as a structural ceiling. Table 7 shows the joint behaviour of failure rate and DBpedia chain connectivity on MuSiQue. Connectivity drops sharply with hop count and the failure rate climbs accordingly; at 4-hop with intermediate bridge type "4hop2", connectivity reaches 0% and the system fails on 92.6% of questions ($n = 27$, small sample). Past two hops, DBpedia connectivity collapses on these benchmarks; prompt-level changes are unlikely to recover this regime without KG expansion or stronger external retrieval.

What is *not* the bottleneck. Two mechanisms are not the bottleneck—entity linking and final-synthesis reasoning—even though both could plausibly be. Of the 8 HotpotQA Cluster VII failures, none stem from entity linking; all are null sub-question outputs from downstream LLM failure. Entity linking is also absent from 2Wiki and MuSiQue top causes; the Spotlight + SPARQL fallback chain does not appear among the top failure causes. Final-synthesis reasoning is also a minor contributor: only 7/82 MuSiQue Cluster II cases are true synthesis errors.

Caveats. All counts use GEMMA3:12B as judge, which is more lenient than strict EM; a stricter judge would shift Cluster II and VI counts upward. The 4hop2 regime ($n = 27$) is small; we treat the 92.6% figure as indicative rather than conclusive. One untracked failure mode—final-synthesis hallucination overriding a literal "`Unknown`" sub-question answer with parametric memory—is currently absorbed into Cluster II (≥ 1 confirmed; up to 16 MuSiQue cases involve sub-questions returning literal "`Unknown`" where override is plausible but unverified).

In summary, four findings drive future work, paired by what they implicate. The KG draws

a boundary: (1) 40% of failures arise when the KG forces a text fallback that fails for label-, retrieval-, or chain-disconnect reasons; (4) on MuSiQue, hop depth dictates outcome through DBpedia connectivity, defining a ceiling that prompt-level changes alone cannot lift without KG expansion. Routing leaks inside that boundary: (2) decomposition errors are bidirectional, with both under- and over-decomposition observable; (3) routing bypass on multi-hop questions is the cheapest universal fix (65 cases, single guard).